

El lenguaje SQL



SQL es el lenguaje por excelencia de definición, manipulación y control de bases de datos relacionales (tablas).



Es un lenguaje declarativo:
expresamos lo que se quiere hacer.



Elementos fundamentales:

Tablas: estructuramos los datos en tablas.

Columnas: pueden ser de tipo carácter, numérico, fecha, entre otros.

Filas: diferentes registros (i.e. clientes, operaciones, expedientes,...).

Tipos de Datos

Tipo	Descripción	Ejemplos
STRING	Datos de caracteres de longitud variable.	Perez', '34', '1.9'
BYTES	Datos binarios de longitud variable.	
INTEGER(o INT64)	Enteros de 64 bits.	6; 1000
FLOAT(o FLOAT64)	Valores numéricos de doble precisión.	6,02; 1000,00
NUMERIC	Valores decimales con una precisión de hasta 38 dígitos y 9 decimales	
BIGNUMERIC	Valores decimales con una precisión de hasta 76.76 dígitos.	
BOOLEAN(o BOOL)	Valores booleanos (TRUE o FALSE).	True, False
DATE	Fecha en el calendario gregoriano (sin zona horaria).	1-1-24
DATETIME	Fecha y hora (sin zona horaria).	1-1-24 0:00
TIME	Hora del día (sin fecha ni zona horaria).	0:00:00
TIMESTAMP	Marca de tiempo que representa un punto absoluto en el tiempo.	2020-06-02 23:57:12.120174 UTC

Sentencias de Definición de Datos: DDL

- **CREATE:** creación de nuevas tablas.

```
CREATE TABLE Atletas (  
  PK_PERSONA      INT NOT NULL  
  Nombre          VARCHAR(50) NOT NULL,  
  Estatura        DECIMAL(2, 1) NOT NULL,  
  Peso            DECIMAL(5, 2),  
  Genero          VARCHAR(20),  
  Nacionalidad    VARCHAR(20),  
  PRIMARY KEY (PK_PERSONA) );
```

```
CREATE TABLE nombre_de_la_tabla (  
  nombre_columna1 tipo_dato [CONSTRAINT],  
  nombre_columna2 tipo_dato [CONSTRAINT], ...  
  PRIMARY KEY (nombre_columna1) );
```

```
CREATE TABLE clientes_valencia AS  
SELECT * from clientes  
WHERE cod_provincia = 46;
```

[CONSTRAINT]: Restricciones opcionales para las columnas (por ejemplo, NOT NULL, UNIQUE, DEFAULT).

Sentencias de Definición de Datos: DDL

- **ALTER:** modificación de la estructura de tablas existentes:

- Añadir una columna nueva:

```
ALTER TABLE nombre_de_la_tabla ADD nombre_columna  
tipo_dato [CONSTRAINT];
```

- Modificar una columna existente:

```
ALTER TABLE nombre_de_la_tabla MODIFY nombre_columna  
tipo_dato [CONSTRAINT];
```

- Renombrar una columna existente:

```
ALTER TABLE nombre_de_la_tabla RENAME COLUMN  
nombre_columna_anterior TO nombre_columna_nuevo
```

Sentencias de Definición de Datos: DDL

- **ALTER:** modificación de la estructura de tablas existentes:

- Borrar una columna existente:

```
ALTER TABLE nombre_de_la_tabla DROP COLUMN  
nombre_columna
```

- Añadir una restricción:

```
ALTER TABLE nombre_de_la_tabla  
    ADD CONSTRAINT nombre_restricción tipo_restricción  
(nombre_columna);
```

Definición de Datos: DDL

■ **ALTER:** Ejemplos:

- Añadir una columna nueva:
- Modificar una columna existente:
- Renombrar una columna existente:
- Borrar una columna existente:
- Añadir una restricción:

```
ALTER TABLE Atletas  ADD Email VARCHAR(100);
```

```
ALTER TABLE Atletas  MODIFY Estatura DECIMAL(3, 2)  
NOT NULL;
```

```
ALTER TABLE Atletas  
  RENAME COLUMN Estatura TO Altura;
```

```
ALTER TABLE Atletas  DROP COLUMN Nacionalidad ;
```

```
ALTER TABLE Atletas  ADD CONSTRAINT FK_Categoria  
FOREIGN KEY (FK_Categoria ) REFERENCES  
Categorias(PK_PERSONA );
```

Manipulación de Datos: DML

Tabla **Atletas**

pk_persona	Nombre	Estatura	Peso	Género
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

Seleccionamos todas las filas:

```
SELECT *  
FROM Atletas;
```

Manipulación de Datos: DML - SELECT

SELECT *
FROM Atletas;

↑
FROM: indicamos la tabla/s de donde sacaremos la información.

←
SELECT : Selección de variables (columnas) que queramos visualizar.

TIP: si usas * seleccionas todas las columnas de la tabla

SELECT pk_persona AS id, Estatura, Peso
FROM Atletas;

→ Sólo se seleccionan las variables que interesan

Manipulación de Datos: DML - SELECT

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	Carlos	159	56	Mujer
3	Pedro	176	80	Hombre
4		165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

Para contar los registros no nulos

```
SELECT COUNT(Nombre)  
FROM Atletas;
```

→ Resultado: 8

Para contar registros no nulos DIFERENTES

```
SELECT COUNT(DISTINCT Nombre)  
FROM Atletas;
```

→ Resultado: 7

Para contar todos los registros

```
SELECT COUNT(*)  
FROM Atletas;
```

→ Resultado: 9

Manipulación de Datos: DML - WHERE

Usaremos WHERE como un filtro o para imponer condiciones.

Queremos sacar los resultados de la tabla para aquellos registros con un peso superior o igual a 80

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

```
SELECT *  
FROM DATOS_PERSONAS  
WHERE Peso >= 80;
```

Manipulación de Datos: DML - WHERE

Operadores

Operador	Acción
=	Igualdad
<>	Distinto de
>	Mayor
>=	Mayor o igual
<	Menor
<=	Menor o igual
IS NULL	Selecciona únicamente los valores nulos (NULL).
IS NOT NULL	Selecciona todos los valores no nulos (NULL)
BETWEEN	TRUE si el valor indicado está dentro de un intervalo.
IN	TRUE si el valor indicado es igual a uno dentro de una lista de expresiones.
LIKE	TRUE si el el valor coincide con un patrón.
EXISTS	TRUE si una subconsulta contiene cualquiera de los valores indicados

Manipulación de Datos: DML - WHERE

Se pueden usar intervalos en el comando WHERE usando BETWEEN.

Queremos sacar los resultados de la tabla para aquellos registros con un peso entre 70 y 80

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

```
SELECT *  
FROM Atletas  
WHERE Peso BETWEEN 70 AND 80;
```

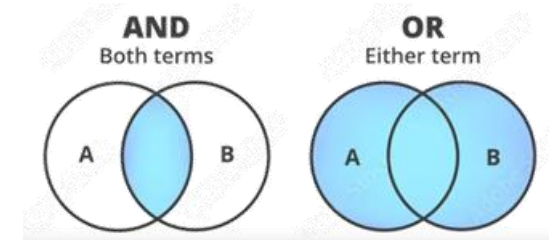
Manipulación de Datos: DML - WHERE

¿Y si buscamos los hombres cuyo peso sea mayor que 80?

Usaremos AND o OR en el comando WHERE para añadir más condiciones

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

```
SELECT *  
FROM Atletas  
WHERE Peso > 80 AND Genero='Hombre';
```



AND: ambas condiciones deben cumplirse.

OR: cualquiera de las dos (o ambas) deben cumplirse

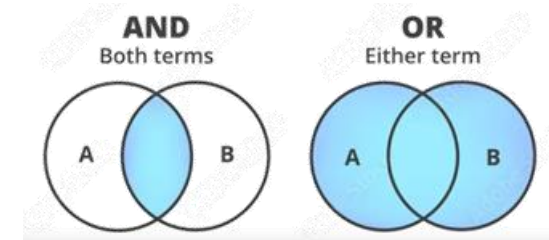
Manipulación de Datos: DML - WHERE

¿Y si necesito filtrar la tabla con sólo aquellos con peso mayor a 80 y hombres o estatura menor a 160 y mujeres?

Simplemente intercalas un “AND” o “OR” entre cada condición.

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

```
SELECT *  
FROM Atletas  
WHERE (Peso>80 AND Genero='Hombre') OR  
(Estatura<160 AND Genero='Mujer');
```



AND: ambas condiciones deben cumplirse.

OR: cualquiera de las dos (o ambas) deben cumplirse

Manipulación de Datos: DML - WHERE

Y ¿si necesito filtrar la tabla con sólo aquellos con peso mayor a 80 y hombres o estatura menor a 160 y mujeres?

```
SELECT *  
FROM Atletas  
WHERE (Peso > 80 AND Genero='Hombre') OR  
(Genero='Mujer' AND Estatura<160);
```

Y ¿si necesito filtrar la tabla con sólo con aquellos con altura mayor a 160 ó que pesen más de 70?

```
SELECT *  
FROM Atletas  
WHERE Estatura > 160 OR Peso > 70
```

Se pueden agregar tantas condiciones como se necesiten. Sólo se debe considerar utilizar () para agrupar condiciones.

Manipulación de Datos: DML - WHERE

Y ¿si necesito filtrar la tabla sólo con María, Juan y Samuel?

2 formas: usando OR o IN

```
SELECT *  
FROM Atletas  
WHERE Nombre = 'María' OR Nombre='Juan' OR  
Nombre='Samuel';
```

```
SELECT *  
FROM Atletas  
WHERE Nombre IN ('María','Juan','Samuel');
```

Manipulación de Datos: DML – FUNCIONES

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

```
SELECT COUNT(pk_persona)  
FROM Atletas;
```



Resultado: 9

Manipulación de Datos: DML – FUNCIONES

¿Qué sucede si la columna “Nombre” tiene datos vacíos o nulos?

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4		165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9		176	65	Mujer

Si usamos COUNT, no considera los nulos

```
SELECT COUNT(pk_persona)
FROM Atletas;
```

```
SELECT COUNT(*)
FROM Atletas;
```

Ejemplos de queries para detectar columnas o registros (filas) con nulls.

```
SELECT COUNT(*)
FROM Atletas
WHERE Nombre IS NULL;
```

```
SELECT *
FROM Atletas
WHERE Nombre IS NULL ;
```

Manipulación de Datos: DML – FUNCIONES

¿Y cómo puedo sumar las estaturas de las personas con peso menor o igual a 70?

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

```
SELECT SUM(Estatura)
FROM Atletas
WHERE Peso <= 70;
```

Resultado: 826

```
SELECT MAX(Estatura)
FROM Atletas
WHERE Peso <= 70;
```

Resultado: ?

```
SELECT MIN(Estatura)
FROM Atletas
WHERE Peso <= 70;
```

Resultado: ?

```
SELECT AVG(Estatura)
FROM Atletas
WHERE Peso <= 70;
```

Resultado: ?

Son funciones numéricas.

Manipulación de Datos: DML – GROUP BY

¿Y cómo puedo contar la cantidad de personas por género? Para esto usamos **GROUP BY**

```
SELECT GENERO, COUNT(PK_PERSONA) AS NUM_PERS  
FROM Atletas A  
GROUP BY GENERO;
```

Resultado



GENERO	NUM_PERS
Hombre	6
Mujer	3

```
SELECT Genero, COUNT(pk_persona), SUM(Estatura)  
FROM Atletas  
WHERE ESTATURA < 180  
GROUP BY Genero;
```

Se pueden combinar diferentes comandos, pero siempre manteniendo la estructura (SELECT, FROM, WHERE, GROUP BY)

ORDER BY nos permite ordenar la tabla resultante por la variable que se le indique. Agregando ASC o DESC, podemos indicar el orden a utilizar (ascendente o descendiente)

Manipulación de Datos: DML – GROUP BY

¿Y si además de contar, quiero calcular el promedio de la estatura por género?

Basta agregar la **AVG()** en el **SELECT**

```
SELECT Genero, COUNT(pk_persona), AVG(Estatura)
FROM Atletas
GROUP BY Genero;
```

RESUMEN DE FUNCIONES ÚTILES

- **COUNT()**: cuenta el número de registros en cada grupo.
- **SUM()**: suma el valor de la columna para todos los registros en cada grupo.
- **AVG()**: promedia el valor de la columna para todos los registros de cada grupo.
- **MAX()**: toma el máximo de la columna en cada grupo.
- **MIN()**: toma el mínimo de la columna en cada grupo.
- **PERCENTIL_DISC(XX) WITH IN GROUP (ORDER BY variable)** → obtiene el percentil XX dentro del grupo ordenado por la variable (por defecto es orden ascendente)

Manipulación de Datos: DML – CASE WHEN

¿Y si quiero agrupar por tipo de estatura y generar algo así?

pk_persona	Nombre	Estatura	Peso	Genero	TIPO_ESTATURA
1	Carlos	174	76	Hombre	Medio
2	María	159	56	Mujer	Bajo
3	Pedro	176	80	Hombre	Medio
4	Camila	165	63	Mujer	Medio
5	José	188	86	Hombre	Alto
6	Juan	191	96	Hombre	Alto
7	Bernardo	162	58	Hombre	Medio
8	Samuel	164	70	Hombre	Medio
9	Claudia	176	65	Mujer	Medio

Para ello, primero que todo, se debe tener una definición de “tipo de estatura”.

Imaginemos que queremos 3 tipos:

- Altos: personas con estatura mayor o igual 180
- Medios: personas con estatura entre 160 y 180
- Bajos: personas con estatura menor o igual a 160

Para esto, se debe utilizar la sentencia **CASE WHEN THEN ELSE END**

Manipulación de Datos: DML – CASE WHEN

Es una construcción condicional que permite realizar evaluaciones y tomar decisiones en función de condiciones especificadas. Puedes utilizar esta sentencia para realizar transformaciones en tus datos o para generar resultados basados en condiciones específicas.

SELECT

```
CASE WHEN CONDITION_1 THEN RESULT_1  
WHEN CONDITION_2 THEN RESULT_2  
WHEN CONDITION_3 THEN RESULT_3
```

...

```
ELSE ELSE_RESULT  
END
```

```
FROM TABLA
```

WHEN condition_1: define la primera condición a evaluar
THEN condition_1: Especifica el resultado si la condición 1 es verdadera.

ELSE else_result: Especifica el resultado si ninguna de las condiciones anteriores es verdadera (opcional).

El comando **CASE** inicia la sentencia y **END** indica el final de la sentencia

Manipulación de Datos: DML – CASE WHEN

En el ejemplo anterior,

pk_persona	Nombre	Estatura	Peso	Genero	TIPO_ESTATURA
1	Carlos	174	76	Hombre	Medio
2	María	159	56	Mujer	Bajo
3	Pedro	176	80	Hombre	Medio
4	Camila	165	63	Mujer	Medio
5	José	188	86	Hombre	Alto
6	Juan	191	96	Hombre	Alto
7	Bernardo	162	58	Hombre	Medio
8	Samuel	164	70	Hombre	Medio
9	Claudia	176	65	Mujer	Medio

```
CREATE TABLE TABLA_01 AS
SELECT *,
      CASE WHEN Estatura >= 180 THEN 'Alto'
      WHEN Estatura > 160 AND Estatura < 180 THEN 'Medio'
      WHEN ESTATURA<=160 THEN 'Bajo'
      ELSE 'Sin Dato'
      END AS TIPO_ESTATURA
FROM Atletas;
```

Debemos darle un nombre a la nueva definición que hemos creado, en este caso la llamamos “TIPO_ESTATURA” y usaremos el comando **AS**

Manipulación de Datos: DML – CASE WHEN

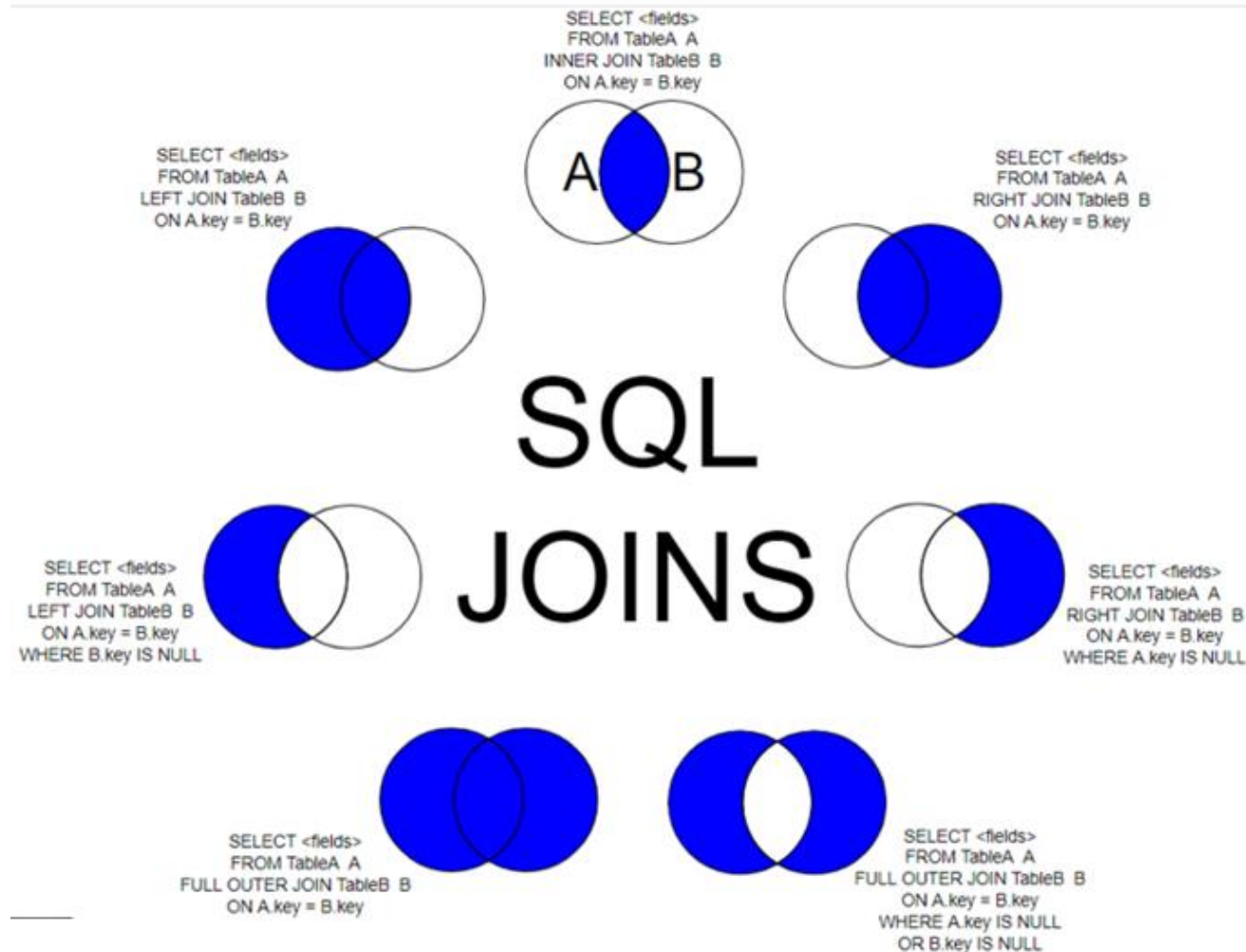
¿ Y si quiero agrupar por tipo de estatura?

```
SELECT TIPO_ESTATURA,  
       COUNT(pk_persona) AS num,    AVG(Estatura) AS estatura_media  
FROM TABLA_01  
GROUP BY TIPO_ESTATURA;
```

Si queremos codificarlo en una misma consulta

```
SELECT CASE WHEN Estatura >= 180 THEN 'Alto'  
           WHEN Estatura > 160 AND Estatura < 180 THEN 'Medio'  
           WHEN Estatura <= 160 THEN 'Bajo'  
           ELSE 'Sin dato'  
           END AS TIPO_ESTATURA,  
       COUNT(pk_persona) as num,  
       AVG(Estatura) as estatura_media  
FROM Atletas  
GROUP BY TIPO_ESTATURA  
ORDER BY TIPO_ESTATURA;
```

Manipulación de Datos: DML – JOINS



Manipulación de Datos: DML – JOINS

TABLE 1

Products	Price
Kiwis	\$6
Onions	\$3
Tomatoes	\$7

TABLE 2

Products	Quantity
Kiwis	10
Onions	6
Broccoli	5



Products	Price	Quantity
Kiwis	\$6	10
Onions	\$3	6

```
SELECT A.*, B.Quantity  
FROM TABLE1 A  
INNER JOIN TABLE2 B  
ON A.Products=B.Products
```



Solo tomas las comunes a través de la variable
"Products" de ambas tablas.

Manipulación de Datos: DML – JOINS

TABLE 1

Products	Price
Kiwis	\$6
Onions	\$3
Tomatoes	\$7

TABLE 2

Products	Quantity
Kiwis	10
Onions	6
Broccoli	5



Products	Price	Quantity
Kiwis	\$6	10
Onions	\$3	6
Tomatoes	\$7	Null

```
SELECT A.*, B.Quantity  
FROM TABLE1 A  
LEFT JOIN TABLE2 B  
ON A.Products=B.Products
```

1. A la tabla 1 añade el atributo Quantity. En caso de no existir coincidencia se pone NULL.
2. Ignora filas de la tabla 2 que no estén en la tabla 1.

Manipulación de Datos: DML – JOINS

Tabla PEDIDO

Products	Quantity	Category
Kiwis	10	VIP
Onions	6	Standard
Tomatoes	5	Standard
Jamón	1	5J

Tabla PRICES

Products	Category	Price
Kiwis	Standard	10
Onions	Standard	6
Broccoli	Standard	5
Kiwis	VIP	15
Onions	VIP	8
Broccoli	VIP	9
Tomatoes	VIP	10
Jamón	Serrano	20
Jamón	5J	100

```
SELECT A.*, B.Price, A.Quantity*B.Prices AS Total_Product  
FROM PEDIDO A
```

```
LEFT JOIN PRICES B
```

```
ON A.Products=B.Products AND A.Category = B.Category
```



En este JOIN, buscamos la coincidencia tanto en la columna Products como en Category.

En el caso de Tomatoes VIP no encuentra coincidencia y devuelve NULL.

Además, se ha calculado el coste multiplicando cantidad y precio.



Products	Quantity	Category	Price	Total_Product
Kiwis	10	VIP	15	150
Onions	6	Standard	6	36
Tomatoes	5	Standard	NULL	NULL
Jamón	1	5J	100	100

Manipulación de Datos: DML – JOINS

TABLE 1

Products	Price
Kiwis	\$6
Onions	\$3
Tomatoes	\$7

TABLE 2

Products	Quantity
Kiwis	10
Onions	6
Broccoli	5



Products	Price	Quantity
Kiwis	\$6	10
Onions	\$3	6
Broccoli	Null	5
Tomatoes	\$7	Null

```
SELECT A.*, B.Quantity  
FROM TABLE1 A  
FULL JOIN TABLE2 B  
ON A.Products=B.Products
```

Manipulación de Datos: DML – JOINS

Consideremos las siguientes dos tablas:

Tabla Atletas

pk_persona	Nombre	Estatura	Peso	Genero
1	Carlos	174	76	Hombre
2	María	159	56	Mujer
3	Pedro	176	80	Hombre
4	Camila	165	63	Mujer
5	José	188	86	Hombre
6	Juan	191	96	Hombre
7	Bernardo	162	58	Hombre
8	Samuel	164	70	Hombre
9	Claudia	176	65	Mujer

Tabla COMPRAS (cada registro representa una compra)

pk_persona_ID	pk_anyomes	imp_compra	num_articulos
1	202001	35	2
1	202002	67	3
1	202004	78	1
2	202001	43	2
2	202002	56	2
3	202004	99	1
4	202001	37	4
4	202002	102	6
4	202003	150	3
4	202004	120	2
5	202004	56	2
5	202005	97	3
6	202002	150	4
6	202003	190	6
6	202005	98	7
7	202001	88	2
7	202003	79	4
7	202004	49	2
7	202005	230	9
8	202003	178	2
9	202004	56	1
9	202005	37	2

Manipulación de Datos: DML – JOINS

¿Cómo puedo agregar el nombre de la persona en la tabla COMPRAS?

Necesitamos que a la tabla COMPRAS se le agregue la variable “Nombre”, que se encuentra en la tabla Atletas, por lo tanto, podemos realizar un LEFT JOIN

Hands on

PASO 1: Definir orden de las tablas

PASO 2: Identificar la PK (Primary Key) o variable de referencia para hacer la unión de cada tabla

PASO 3: Seleccionar las variables que queremos que aparezcan en la consulta (SELECT)